

When Delivery Meets Error: Exploring Email Delivery Retry Strategies and Defects

Ruixuan Li*
Tsinghua University
Beijing, China
lirx25@mails.tsinghua.edu.cn

Baojun Liu
Tsinghua University
Beijing, China
lbj@tsinghua.edu.cn

Enlong Li*
Quan Cheng Laboratory
Jinan, China
ts-liel@qcl.edu.cn

Haixin Duan
Quan Cheng Laboratory
Jinan, China
Tsinghua University
Beijing, China
duanhx@tsinghua.edu.cn

Mingxuan Liu
Zhongguancun Laboratory
Beijing, China
liumx@mail.zgclab.edu.cn

Qingfeng Pan
Coremail Technology Co. Ltd
Guangdong, China
pqf@coremail.cn

ABSTRACT

Reliable delivery is a basic expectation of email services. The increasingly complex ecosystem has led to more frequent email bounces. When encountering various errors, properly handling failures and retrying are crucial to ensuring email deliverability. Currently, the community lacks a comprehensive understanding of email retry strategies, and bridging this gap can enhance the reliability of email services.

In this paper, we measure the email delivery retry strategies of 11 email service providers and 8 open-source email software. We deploy a platform (RetryWatch) that simulates various types of bounce errors returned by email receiving servers, covering the DNS, TCP, and SMTP communication stages. We find that many providers and software exhibit defects in their email retry implementations. For example, we find that six email systems exhibit poor load balancing across MX record entries in dual-stack environments. In addition, when encountering blacklist- and greylist-related SMTP error messages, most email providers fail to handle retries appropriately. We release RetryWatch to support further testing by the email community. We hope this work helps improve email reliability and inspires efforts to refine email retry strategies.

CCS CONCEPTS

• **Networks** → **Application layer protocols**; **Network reliability**; • **Security and privacy** → **Security protocols**.

KEYWORDS

Internet measurement; Email delivery retry; SMTP; DNS

ACM Reference Format:

Ruixuan Li, Enlong Li, Mingxuan Liu, Baojun Liu, Haixin Duan, and Qingfeng Pan. 2026. When Delivery Meets Error: Exploring Email Delivery Retry

*Ruixuan Li and Enlong Li contributed equally to this work.



This work is licensed under a Creative Commons Attribution International 4.0 License.

IMC '26, October 12–16, 2026, Karlsruhe, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2327-8/2026/10
<https://doi.org/10.1145/3777912.3839798>

Strategies and Defects. In *Proceedings of the 2026 ACM Internet Measurement Conference (IMC '26)*, October 12–16, 2026, Karlsruhe, Germany. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3777912.3839798>

1 INTRODUCTION

Email is one of the most important means of online communication and identity verification [16]. With the rapid evolution of email service models and the continuous introduction of new security mechanisms, the email ecosystem has become increasingly complex [8]. This growing complexity introduces potential drawbacks, most notably an increase in email delivery failure rates. In 2024, Li et al. [11] reported that, among 298 million emails delivered by a large email provider over a 15-month period, 13% experienced delivery failures, and one-third of those could be successfully delivered after retries.

Deliverability has always been the most fundamental expectation of the email community regarding email services. Therefore, when facing various email bounce scenarios, properly handling delivery failures is crucial to the deliverability of email services. Currently, Internet standards do not specify the implementation details of email retry strategies. As a result, email providers and software systems typically handle delivery failures based on their own interpretations.

However, the community lacks a comprehensive understanding of email retry strategies and their defects. Holzbauer et al. [8] conducted preliminary tests on email retry strategies under greylisting scenarios. Li et al. [11] only provided a brief description of the retransmission policy used by their partner provider. Bridging the knowledge gap on email delivery retry strategies can help improve email deliverability.

Our study. This paper presents a measurement study of email retry implementations, focusing on how servers handle emails after encountering different bounce errors. To accomplish this, we designed a controlled experimental platform (RetryWatch), which returns delivery failures under various conditions by deploying controlled receiving servers. This enables us to observe and compare the responses of different systems, such as retry behavior, frequency, and server selection. Based on the email delivery process, we primarily consider three types of delivery failure scenarios: DNS resolution, TCP connection, and SMTP communication. In total, we designed

60 test cases, implemented through different subdomains of our controlled domains. In this study, we evaluated 19 email systems: we sent emails to RetryWatch from 11 email service providers and 8 open-source email software to assess their retry strategies.

We find that six email systems fail to achieve effective load balancing across equal-priority MX records in dual-stack environments, instead using only IPv4 or IPv6 for delivery. In addition, many email systems exhibit retry strategies that conflict with the semantics of security mechanisms. Specifically, when encountering blacklist-related errors, switching IP addresses to retry can improve delivery success rates; however, eight email providers frequently retry using the same IP address or do not perform email retries. In contrast, when encountering greylist-related errors, email systems should retry the email using the same IP address. Unfortunately, we observe that nine providers switch IP addresses in such cases, which inevitably leads to delivery failure.

Contributions. Contributions of this paper are as follows:

- We present the first exploration of email retry strategies, revealing a series of flaws, including violations of load-balancing principles and conflicts with security mechanisms.
- We make our experimental platform publicly available (<https://www.retrywatch.email>), allowing researchers to test email retry strategies online.

2 BACKGROUND AND RELATED WORK

2.1 Email Delivery and Retry

We first introduce the classical email delivery process. After receiving an email from the sender, the sending server resolves the MX records of the recipient domain to determine the delivery destination. The sending server then establishes the TCP and SMTP sessions with the receiving server. Finally, the receiving server delivers the email to the recipient.

The email delivery process involves many network protocols and security mechanisms, and any failure in these components can lead to delivery failure. To ensure the availability of email services, sending servers typically attempt to re-deliver bounced emails. In particular, email is a delay-tolerant communication service, which allows multiple and complex retry mechanisms to be accepted by the community.

Figure 1 illustrates the macroscopic process of email delivery retries, which involves DNS resolution, server selection, TCP connection, and SMTP sessions. First, the sending server queries the MX records of the recipient domain and the corresponding A or AAAA records through DNS. If the query fails (e.g., SERVFAIL), the sending server performs DNS retries. After obtaining the IP addresses of the incoming servers, the sending server selects one IP address for connection based on its own strategy. These strategies may consider factors such as MX priority and IP address type.

Next, the sending server attempts to establish a TCP connection and an SMTP session with the receiving server. The TCP connection involves a three-way handshake between peer servers. The SMTP session consists of exchanging SMTP commands and SMTP responses. Common SMTP commands include HELO, MAIL FROM, RCPT TO, DATA, and QUIT. SMTP responses are used to inform the sending server of the execution status of these commands. For example, “200 OK” indicates that a command has been successfully

recognized and executed, and the session can proceed. In addition, the receiving server provides the reason for delivery failure through a Non-Delivery Report (NDR) [16, 25]. An NDR typically consists of three components: a reply code (e.g., 452), an enhanced mail system status code (e.g., 4.2.2), and a detailed error message (e.g., sender hit blacklist).

During the above process, if an error occurs, the sending server decides whether to continue retrying or terminate the process based on its retry policy. If it continues retrying, the email is placed into the retry queue of the sending server and waits for the next delivery attempt.

2.2 Related Work

For email delivery services, prior research has primarily focused on delivery errors and outbound configuration, while paying little attention to the strategies and behaviors of email retries. Li et al. [11] analyzed 15 months of email delivery logs from a large email provider, revealing the overall delivery failure rate and systematically examining the causes of bounced messages. They reported that 13% of emails experience a bounce, and roughly one-third of these messages are successfully delivered after retries. These findings highlight the critical role of retry mechanisms in real-world email deliverability. Notably, Li et al. [11] also briefly described the delivery strategy of their partner provider and pointed out its incompatibility with greylisting. However, their study does not provide an in-depth analysis of email retry mechanisms.

Furthermore, Holzbauer et al. [8] studied the basic operational properties of email delivery services by recruiting volunteers to send emails, such as IP support, TLS configuration, DNSSEC validation, and greylisting effectiveness. Their study primarily focuses on individual email delivery attempts rather than the behaviors following delivery failures. Ruohonen et al. [20] further analyzed fundamental characteristics of domain configurations, including the number of MX records, their priorities, and associated IP addresses. Researchers have also observed that malicious servers tend to use distinctive delivery state-machine patterns [12], which can assist detection systems in filtering spam.

3 METHODOLOGY

In this section, we introduce RetryWatch, the experimental platform used to measure email retry strategies. We then describe the target email systems evaluated in this paper.

3.1 Implementation of RetryWatch

RetryWatch is designed to act as an email receiving server and can simulate a variety of delivery failure scenarios. To facilitate testing, each scenario is implemented through a distinct subdomain and username under our controlled domain. When a tested system sends an email to a specific email address, RetryWatch returns the corresponding error response, thereby triggering the target system’s retry mechanism. We use BIND9 as the authoritative name server to configure and manage the various subdomains and DNS resource records.

RetryWatch consists of four main functional components: one baseline measurement module and three error-simulation modules. The baseline module evaluates the default delivery behavior of

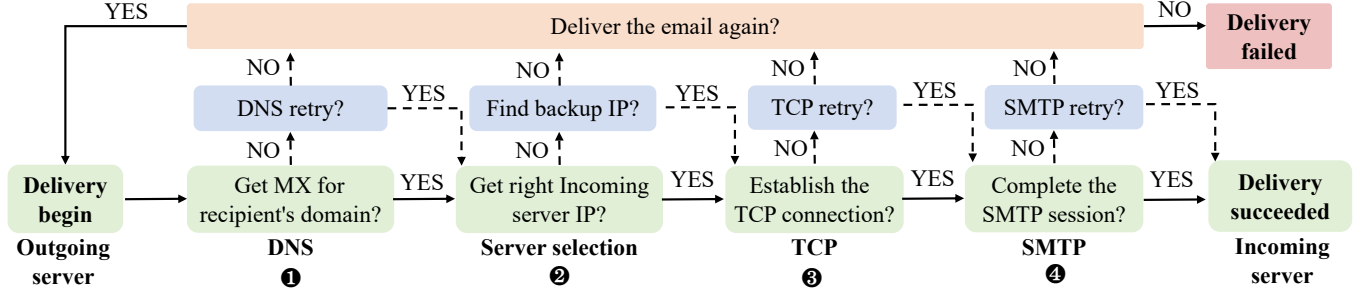


Figure 1: Simplified process of email delivery and retry.

Table 1: The test cases designed for the email retry strategy in this paper.

Description	Type	Subdomains
Baseline	Single stack	tmail, tmail4e, tmail6e, tmail4edual, tmail6edual, tmail4, tmail6
	Dual stack	tmail4hdual, tmail6hdual, tmail64s, tmail64d, tmail64dual, tmail64h4l6, tmail64h6l4
DNS error	Base	nxdomain, refused, servfail, timeout, nonmx, nonda, nondaaaa
TCP error	Base	icmp-dest-unreach, tcp-rst, tcp-slient
SMTP error	NDR	helo-550, helo-552, helo-554, helo-421, helo-450, helo-451, helo-452, mail-550, mail-552, mail-554, mail-421, mail-450, mail-451, mail-452, rcpt-550, rcpt-552, rcpt-554, rcpt-421, rcpt-450, rcpt-451, rcpt-452, data-550, data-552, data-554, data-421, data-450, data-451, data-452
	Timeout	slient-helo, slient-mail, slient-rcpt, slient-data
	Disconnect	close-conn-helo, close-conn-mail, close-conn-rcpt, close-conn-data

target systems under normal reception conditions, serving as a control group to better interpret the retry mechanisms. The three error-simulation modules assess how target systems retry email delivery when failures occur at the DNS, TCP, and SMTP stages. Table 1 lists 60 test cases we have designed. In the following, we describe each module of RetryWatch in detail.

Baseline module. This module observes the default outbound behavior of email systems in a normal network environment, without introducing any failures. We focus on the target systems' default server-selection logic and load-balancing strategies.

The baseline module includes three scenarios: IPv4-only, IPv6-only, and dual-stack. In the IPv4-only scenario, we configure each subdomain with two MX records, and each MX record resolves to three A records. In the IPv6-only scenario, each MX record resolves to three AAAA records. For the dual-stack scenario, each MX record resolves to one A record and one AAAA record. We consider both equal-priority and different-priority configurations for the two MX records.

DNS error simulation module. This module evaluates how target email systems behave when failures occur during the MX lookup of the recipient domain. The test cases we cover include NXDOMAIN, SERVFAIL, and REFUSED responses. In addition, we include a scenario where the domain has no MX records but does have an A/AAAA record. According to RFC 5321, email systems should attempt delivery to the host specified by the A/AAAA record in such cases.

TCP error simulation module. These failure conditions are designed to examine how target email systems retry delivery when

encountering TCP-level issues. We introduce three specific types of failures:

- Port unreachable. When the target system attempts to connect to the SMTP port, RetryWatch returns an ICMP "port unreachable" message.
- Connection reset. RetryWatch sends a TCP RST packet to abruptly terminate the connection.
- Connection timeout. RetryWatch receives the target system's TCP SYN packet but remains silent, causing the connection attempt to time out.

SMTP error simulation module. Based on prior work summarizing the causes of SMTP communication errors [11], we designed a variety of fault scenarios to study how these errors influence retry behavior. We focus on three major types of failures: reply error message, connection timeout, and termination. For each test, RetryWatch returns crafted error responses after receiving different SMTP commands. The commands we test include EHLO, MAIL FROM, RCPT TO, and DATA. In addition, when the target system is transmitting the email body, RetryWatch also returns error responses.

• Non-delivery report. In this paper, we use 12 NDRs to evaluate email systems, covering multiple aspects such as sender reputation, server availability, and recipient mailbox status. These NDRs are derived from public reports and prior research, rather than being arbitrarily constructed. The detailed NDR messages are listed in Appendix A.

• SMTP timeout. After sending an SMTP command, the sending server waits for a response from the receiving server within

a bounded time. If no response is received, the sender will terminate the SMTP session. To emulate this behavior, we configure the incoming email server to remain silent after receiving an SMTP command until the sending server disconnects the SMTP connection. This setup enables us to examine how target systems retry in SMTP timeout scenarios.

- SMTP disconnect. Email delivery is a continuous process, and an unexpected interruption of the SMTP connection results in a delivery failure. To simulate this, we configure the receiving server to immediately and proactively close the SMTP connection after receiving an SMTP command. This allows us to evaluate how target systems handle and retry email when facing SMTP-level connection interruptions.

3.2 Tested Email Systems

The test targets of this study include both public email service providers and open-source email server software. Specifically, we selected 10 public email service providers where we can register accounts: Gmail [6], Outlook [14], QQ [19], 163 [2], Sina [22], Sohu [23], Aliyun [3], Zoho [28], Yandex [27], and Synology [24]. We also tested the email service operated by our university, which is implemented using the software provided by Coremail. For all these providers, we registered legitimate accounts to conduct our experiments.

Furthermore, we tested 8 open-source email software implementations: Exim [5], Postfix [17], Sendmail [21], Qmail [18], Postal [15], Haraka [7], OpenSMTPD [13], and Axigen [4]. All software was deployed locally using their latest available versions in December 2025, running on identical machine and network configurations, and all were tested with default configuration settings.

For public email service providers, emails are sent directly to a mail server under our control. The server is deployed in AS45102 within the 8.210.202.0/24 network. In contrast, experiments involving self-hosted email software are conducted entirely within our local laboratory environment. The sending and receiving mail servers use different IP addresses, and all email traffic remains within the laboratory network without traversing the public Internet. We also deploy a BIND9-based authoritative DNS server in the same subnet as the controlled mail server to provide DNS resolution for the domains used in our experiments. For each test case, we repeat the experiment three times. Unless otherwise specified, the results are consistent across all three runs. Each experiment is conducted within a one-hour testing window. Specifically, during the first hour after an experiment begins, our controlled mail server responds to emails sent to the experimental domain with the corresponding predefined responses, such as SMTP error codes.

4 RESULT

This section first introduces the support of the email system for IP types. Then, we elaborate on the email retry flaws discovered in this paper from four aspects. Table 2 shows the defects in the email retry policies of email providers.

4.1 IP Type Support

We configured the MX records of subdomains with only A or only AAAA records to evaluate the IP support of the target system's outgoing emails. We find that IPv6 support in many email systems is far from ideal: 6 email service providers and 2 open-source email servers fail to operate in an IPv6-only environment. When delivering email to domains that support only IPv6, QQ, 163, Sina, Sohu, and Aliyun return DNS-related errors, such as “unable to resolve MX record” or “DNS lookup failure.” This behavior suggests that these systems may query only IPv4 addresses during the DNS resolution phase. In contrast, Zoho, Synology, and Qmail return “host unreachable” errors. We infer that these systems likely recognize IPv6 addresses correctly, but the IPv6 path is unreachable due to network-related issues on their end.

4.2 Load Balance Implementation

Domains typically achieve inbound email load balancing by configuring multiple MX records with the same priority, which helps relieve pressure on any single email server. We observe that different email systems interpret load balancing inconsistently: some distribute outbound email across MX records, others balance traffic across IP addresses, and some adopt special server-selection strategies specifically in dual-stack scenarios.

Specifically, we configured domains with two MX records of equal priority, where each MX record resolved to three A records. Under this setup, OpenSMTPD and Haraka perform outbound email load balancing only at the MX-record level. They select a single IP address from each MX record and do not utilize all available IP addresses. This results in the expected email traffic load distribution not being fully achieved. In contrast, other email systems utilize all available IP addresses to achieve load balancing.

In the dual-stack scenario, each MX record was configured with both an IPv4 and an IPv6 address. Among email systems that support dual-stack operation, five (Exim, Sendmail, OpenSMTPD, Haraka, and Axigen) rely exclusively on IPv6 addresses for outbound load distribution, ignoring IPv4 addresses. In contrast, our university's email system uses only the IPv4 addresses in the MX records, rather than the IPv6 ones, even though it is capable of sending mail over IPv6. Therefore, when a dual-stack recipient domain communicates with the aforementioned mail systems, it can only release a portion of its traffic load distribution capability. The remaining dual-stack-capable email systems utilize all available IPv4 and IPv6 addresses for outbound email delivery.

4.3 Conflicts with Security Mechanisms

Blocklist. Email blocklists are collections of malicious servers. During SMTP communication, receiving servers query these blocklists and terminate connections with listed peers [9]. Blocklist is an important reason for email bounces in the real world. From the perspective of email accessibility, upon receiving error responses indicating that an IP address is blacklisted, sending email systems should switch to other IP addresses to retry delivery¹. If an email system repeatedly attempts delivery using blocked IP addresses, it

¹Since open source software does not support multiple sending IPs by default, we exclude them when testing blocklist and greylist related errors.

Table 2: Statistics on the defects of the email retry strategy of 11 email providers.

	Gmail	Outlook	QQ	163	Sina	Sohu	Aliyun	Zoho	Yandex	Synology	Our Univ.
IPv6	●	●	○	○	●	○	○	●	●	●	●
Load balance	●	●	●	●	●	●	●	●	●	●	○
Blocklist											
550	○	●	●	●	○	○	○	○	○	○	○
450	●	●	●	●	●	●	●	○	●	●	○
Greylist	○	○	○	○	○	○	○	●	○	○	●

will not only fail to complete delivery and degrade user experience, but also waste network resources.

In our experiments, we instructed the receiving server to return blocklist-related error responses after each SMTP command, in order to observe whether sending email systems would switch IP addresses and retry delivery. Based on prior reports and studies, our tests included two common NDR with different reply codes: “550 5.7.1 Service unavailable, Client host blocked using Spamhaus” and “450 4.7.1 Service unavailable; Client host blocked using Spamhaus”.

We find that when a 450 reply code is returned, the vast majority of email systems attempt to resend the email. However, Zoho and our university frequently retry using the same IP address, while other email systems switch to different IPs. When a 550 reply code was returned, only Outlook, QQ, and 163 switched IP addresses to retry sending email. Other email systems directly determine that the mail delivery has failed and do not perform any retries. This is likely because 5xx reply codes are defined as permanent failures [16], leading email systems to treat the delivery as unsuccessful.

Sender IP addresses being listed on blocklists are a common cause of email delivery failures. Prior studies have shown that approximately one-third of delivery failures are associated with blocklists [11] and that hundreds of thousands of domains deploy blocklist-based filtering [10]. Therefore, inappropriate handling of blocklist-related failures can significantly affect email deliverability.

To improve email deliverability, we provide the following recommendations. For receiving servers, we recommend using 4xx response codes, which indicate temporary failures, for blocklist-related rejections. This allows sending servers to retry the failed delivery rather than treating it as a permanent failure. For sending servers, we recommend retrying the delivery from an alternative IP address when possible. Since a listed IP address is unlikely to be removed from a blocklist within a short period, repeatedly retrying from the same IP address after a short interval, such as five minutes, is unlikely to improve the chance of successful delivery.

Greylisting. Greylisting is a widely used anti-spam mechanism that rejects the first delivery attempt of a specific tuple (sender MTA, sender address, recipient address), and accepts subsequent retries of the same tuple [1]. Prior studies have reported that approximately 3% of email delivery failures are associated with greylisting [11]. To ensure the successful delivery of legitimate emails, the email system encountering a Greylisting error should resend the rejected email using the same sending server.

Based on public realization [1, 11], we find that most Greylisting related error messages begin with a 4xx reply code and instruct the

sender to retry after a fixed delay (e.g., 5 minutes). In our experiments, we configured RetryWatch to return “450 4.2.0 Recipient address rejected: Greylisted. Please try again after 5 minutes” to evaluate how different email systems react to Greylisting errors.

The original design of greylisting assumes that legitimate email servers will retry delivery after a certain period of time. However, our results reveal significant conflicts between the retry mechanisms of many email providers and the semantics of Greylisting. Specifically, Gmail, Outlook, QQ, 163, Sina, Sohu, Aliyun, and Yandex Synology switch the outgoing server’s IP address when retrying the bounced emails, rather than reusing the original IP address. This results in a large number of unsuccessful delivery retries, causing unnecessary resource consumption for both sending and receiving servers.

Additionally, we note that none of the email providers adhere to the retry intervals specified in the SMTP error messages; instead, they deliver messages according to their own predefined retry schedules. These findings clearly indicate that mainstream email systems generally do not attempt to interpret the detailed semantics of SMTP errors, but instead rely on a generic retry mechanism. To reduce ineffective delivery attempts, we recommend that the email community standardize SMTP error responses and corresponding retry strategies for common types of email delivery failures.

4.4 Delivery Penalization

In this paper, delivery penalization refers to the behavior where, due to frequent errors or failures from the high-priority MX, the email system skips it and directly sends emails to lower-priority MX servers. This penalization is persistent from the perspective of the recipient domain: during the penalization period, all emails sent by the email system are routed only to low-priority MX servers. This mechanism may increase the risk of DOS attacks. For example, if a high-priority server is attacked and returns errors, subsequent email traffic may be redirected entirely to low-priority servers, potentially overwhelming them. Below, we introduce the penalty phenomenon that we have observed.

SMTP timeout. First, we configured a subdomain with two MX record entries of different priorities. The high-priority MX server remained silent after receiving SMTP commands until the SMTP connection timed out. We evaluated the effect of silence after different SMTP commands. The tested email system sends one email to our subdomain every 20 seconds.

We find that Synology, Postfix, and Axigen trigger delivery penalization when frequent timeouts occur after the RCPT and DATA

commands. Our university’s email system and Exim trigger penalization when frequent timeouts occur after the DATA command. The occurrence of penalization is not always stable. For example, in experiments with Synology, increasing the email sending interval from 20 seconds to 60 seconds prevents the penalization mechanism from being triggered. This suggests that penalization is likely related to the email sending rate. In addition, the time required to trigger penalization varies significantly across email systems. For instance, Axigen triggers penalization at around 300 seconds, whereas Synology does so at around 120 seconds.

TCP disconnect. We observe that some email systems trigger delivery penalization when they frequently encounter TCP interruptions. The measurement methodology is similar to that used for the above SMTP timeout experiments, except that the subdomain is configured to terminate the TCP connection instead of returning an SMTP error response.

The results show that our university’s email system and Exim exhibit delivery penalization behavior. Both email systems bypass the recovered high-priority servers for a period of time and continue delivering emails to lower-priority servers. However, their recovery times differ: our university’s email system typically resumes using the high-priority server after about two minutes, whereas Exim usually takes more than ten minutes.

4.5 Radical Email Retry

Aggressive message retry is an improper implementation that has been shown to be exploitable for constructing DOS attacks [10, 26]. In this paper, we observe that most email systems adopt relatively conservative retry strategies, with gradually increasing retry intervals: from a few seconds initially, to minutes, to hours, and more.

However, we find that Sendmail performs radical retries when delivering emails to domains configured with many MX records of the same priority. This behavior can be exploited by attackers. Specifically, an attacker configures multiple MX records with identical priority for a controlled domain, all resolving to the victim’s IP address. The attacker then triggers the Sendmail server to send emails to the victim-controlled domain (e.g., via bounce mechanism). As a result, the server sends emails to the victim at a high frequency.

In our experiment, we configured 50 MX records with the same priority for a controlled subdomain, each resolving to the same A record (our email server). We then sent one email from each tested email system to our subdomain and returned an error: “451 4.2.1 Internal resource temporarily unavailable.” We observe that, except for Sendmail, all other email systems perform fewer than 10 retries, with large intervals between attempts. In contrast, Sendmail retries approximately once every 1.5 seconds, exceeding 30 retries within one minute. Such radical retry behavior can excessively consume server resources (e.g., the email queue) and introduce a risk of email traffic amplification.

5 DISCUSSION

5.1 Recommendations

Understanding the semantics of SMTP error messages. In this paper, we observe that most email providers do not interpret the specific semantics of SMTP error messages, but instead rely solely

on reply codes (e.g., 4xx, 5xx) to guide retry behavior. However, such a mechanism may conflict with the intended meaning of certain security mechanisms (e.g., blocklists), leading to unsuccessful email delivery.

SMTP errors related to blocklisting and greylisting exhibit clear textual characteristics. For example, greylisting-related messages often contain keywords such as “greylist.” We recommend that email providers explicitly identify and handle these errors. According to prior work by Li et al. [11], blocklist- and greylist-related errors account for approximately one-third of email delivery failures. Therefore, adopting proper strategies can significantly improve email deliverability while reducing unnecessary consumption of network resources. Moreover, given that email services are inherently delay-tolerant, email providers have a sufficiently large time window to decide how to perform retries.

Improving IPv6 implementation. We observe that many email systems fail to operate in IPv6-only network environments. In addition, load-balancing mechanisms exhibit deficiencies in dual-stack deployments: some email systems select only a subset of MX servers to distribute traffic. With the continued global adoption of IPv6, we recommend that both email system developers and researchers revisit IPv6 implementations, with particular attention to behavioral differences across heterogeneous network environments.

We have reported the identified flaws in email delivery strategies to the relevant organizations, and we are currently in active communication while awaiting their confirmation.

5.2 Limitations

Email delivery error types. In this study, we analyzed the email delivery process and summarized a set of common delivery errors to return to the tested email systems. We acknowledge that these errors do not cover all possible cases. In Appendix B, we further discuss the considerations behind the selection of our test cases.

Tested email systems. Our measurement approach relies on active testing, which limits the evaluated systems to public email providers for which we can register accounts, as well as open-source email software. Therefore, the set of email systems evaluated in our study has certain limitations, as many of them belong to Chinese organizations. Although these systems cover a large portion of mainstream email infrastructure, evaluating private or less widely used systems would also be valuable. In addition, our measurement servers are deployed in only one geographic region. If email systems adopt region-specific retry strategies, this may affect our results.

To broaden the measuring coverage, we release RetryWatch to help researchers and developers to test their email systems. Detailed usage instructions are available at <https://www.retrywatch.email>. We plan to continuously expand the test cases and share new findings in the future. We also released the data generated during the test process at <https://github.com/retrywatch/retrywatch-dataset>.

6 CONCLUSION

In this paper, we designed RetryWatch to conduct the first study targeting email retry policies. By evaluating 11 providers and 8 open-source email systems, we uncover deficiencies in existing retry mechanisms. We hope this study encourages the community to re-examine retry strategies and improve email deliverability.

ETHICS

Our experiments involve a large volume of email transmissions, requiring careful attention to potential ethical risks and mitigation measures.

Experimental environment and targets. For public email providers under test, we use only accounts that we registered ourselves. For open-source email software, we deploy and evaluate them on controlled servers within our laboratory environment. The recipient servers used to simulate various error conditions are also hosted in our lab. Throughout the entire study, no emails are sent to mailboxes or servers outside our control, ensuring that no external users are affected by our experiments.

SMTP communication process. The error messages used in our experiments are all common responses observed during normal SMTP delivery, and do not contain any malicious content or malformed characters. This ensures that our testing does not negatively impact the tested email systems. In addition, we strictly regulate the testing rate to avoid imposing excessive load on target servers. Each test is conducted sequentially, only after the previous one is completed. The frequency and number of retry attempts are entirely determined by the tested email systems themselves; we passively observe their behavior without any active interference.

ACKNOWLEDGMENTS

We thank all anonymous reviewers for their valuable and constructive feedback. This work is supported by the Taishan Scholars Program (tstp20221152) and the National Key Research and Development Program of China (No. 2023YFB3105600). Mingxuan Liu and Baojun Liu are the corresponding authors.

REFERENCES

- [1] 2023. Greylisting.org. <https://www.vpnreactor.com/greylisting-org-acquisition/>.
- [2] 163 Mail. [n. d.]. <https://email.163.com/>.
- [3] Aliyun Mail. [n. d.]. <https://mail.aliyun.com/alimail/auth>.
- [4] Axigen. [n. d.]. <https://www.axigen.com/linux-mail-server/>.
- [5] Exim. [n. d.]. <https://www.exim.org/>.
- [6] Gmail. [n. d.]. <https://gmail.com/>.
- [7] Haraka. [n. d.]. <https://haraka.github.io/>.
- [8] F. Holzbauer, J. Ullrich, M. Lindorfer, and T. Fiebig. 2022. Not that Simple: Email Delivery in the 21st Century. In *USENIX ATC*. USENIX Association, 295–308.
- [9] J. Levine. 2010. DNS Blacklists and Whitelists. RFC 5782.
- [10] R. Li, C. Lu, B. Liu, Y. Lin, Q. Pan, and S. Jun. 2026. CoordMail: Exploiting SMTP Timeout and Command Interaction to Coordinate Email Middleware for Convergence Amplification Attack. In *NDSS*.
- [11] R. Li, S. Xiao, B. Liu, Y. Lin, H. Duan, Q. Pan, J. Chen, J. Zhang, X. Liu, X. Lu, and J. Shao. 2024. Bounce in the Wild: A Deep Dive into Email Delivery Failures from a Large Email Service Provider. In *IMC*. ACM, 659–673.
- [12] P. Lieven, B. Scheuermann, M. Stini, and M. Mauve. 2007. Filtering Spam Email Based on Retry Patterns. In *Proceedings of IEEE International Conference on Communications, ICC 2007, Glasgow, Scotland, UK, 24–28 June 2007*. 1515–1520.
- [13] OpenSMTPD. [n. d.]. <https://github.com/opensmtpd/opensmtpd>.
- [14] Outlook. [n. d.]. <https://outlook.com/>.
- [15] Postal. [n. d.]. <https://github.com/postalserver/postal>.
- [16] J. Postel. 1982. Simple Mail Transfer Protocol. RFC 821.
- [17] Postfix. [n. d.]. <https://www.postfix.org/>.
- [18] Qmail. [n. d.]. <http://qmail.org/>.
- [19] QQ. [n. d.]. <https://mail.qq.com/>.
- [20] J. Ruohonen. 2020. Measuring Basic Load-Balancing and Fail-Over Setups for Email Delivery via DNS MX Records. In *2020 IFIP Networking Conference, Networking 2020, Paris, France, June 22–26, 2020*. IEEE, 815–820.
- [21] Sendmail. [n. d.]. <https://sendmail.org/>.
- [22] Sina Mail. [n. d.]. <https://mail.sina.com.cn/>.
- [23] Sohu Mail. [n. d.]. <https://mail.sohu.com/>.
- [24] Synology Mail. [n. d.]. <https://www.synology.com/en-global/dsm/feature/mailplus>.

- [25] G. Vaudreuil. 2003. Enhanced Mail System Status Codes. RFC 3463.
- [26] W. Xu, X. Li, C. Lu, B. Liu, H. Duan, J. Zhang, J. Chen, and T. Wan. 2023. TsuKing: Coordinating DNS Resolvers and Queries into Potent DoS Amplifiers. In *CCS*. ACM, 311–325.
- [27] Yandex. [n. d.]. <https://360.yandex.com/mail/>.
- [28] ZOHO Mail. [n. d.]. <https://www.zoho.com/mail/login.html>.

A NON-DELIVERY REPORT

In this study, we evaluate the retry strategies of target email systems under common SMTP error conditions by returning Non-Delivery Report (NDR) messages. To obtain representative NDRs, we refer to prior work [11] that categorizes the causes of email delivery failures and collects corresponding NDR messages via search engines. Table 3 summarizes the 12 types of NDRs used in this paper, with an equal split between 5xx and 4xx reply codes.

Table 3: The 12 Non-Delivery Reports (NDRs) used in this paper.

Non-Delivery Report (NDR)
550 5.7.1 Service unavailable, Client host blocked using Spamhaus
554 5.7.0 Email rejected due to security policies
550 5.1.1 Mailbox does not exist
550 5.1.0 This IP has sent too many messages this hour
552 5.2.3 Message size is over limit
550 5.7.1 Message rejected as spam by Content Filtering
450 4.7.1 Service unavailable; Client host blocked using Spamhaus
450 4.2.0 Recipient address rejected: Greylisted. Please try again after 5 minutes
452 4.2.2 The account that you tried to reach is over quota
450 4.2.1 The recipient has exceeded message rate limit
421 4.7.0 This message does not have authentication information or fails to pass checks (SPF or DKIM)
451 4.2.1 Internal resource temporarily unavailable

B SELECTION OF TEST CASES

We design our test cases according to different stages of email delivery, including DNS resolution, TCP connection, and SMTP communication. To evaluate the representativeness of these test cases, we compare them with the statistics of email delivery failures reported by Li et al. [11] in 2024. These bounce reasons are extracted from the real email delivery log of Coremail, a large email service provider in China. The test cases in this paper include SMTP timeouts, DNS resolution errors, blacklist hits, greylisting hits, non-compliant email content, full mailboxes, nonexistent recipients, email authentication failures, and excessive sending rates. According to the measurement results [11], these errors account for more than 80% of all observed email delivery failures.

In addition to representativeness, we ensure that the simulated failures are based on real-world cases rather than arbitrarily constructed errors. DNS and TCP failures generally have standardized and well-defined behaviors. For example, a DNS query may return NXDOMAIN, while a TCP endpoint may respond with an RST

packet. Therefore, our main consideration is the realism of SMTP errors, whose response messages can be customized by individual email providers. To construct realistic SMTP responses, we first refer to the delivery failure types reported by Li et al [11]. We then search Google for common email delivery errors and collect real-world error messages reported in blogs and forums. For example, for blocklist-related errors, we searched Google for “Email bounce reason Spamhaus Microsoft help.” We then found a real-world bounce message reported by an Internet user on a Microsoft forum: “450 4.7.1 Service unavailable; Client host blocked using Spamhaus.” After identifying the delivery failures to simulate, we discussed them with our collaborating provider (Coremail). The provider confirmed that these failures covered common bounce reasons encountered

in real-world email operations and that all of them appeared in its email logs. Although the wording of some bounce messages varied, their underlying meanings were consistent. Finally, when simulating the email delivery failures, our controlled mail server reused the collected error messages.

Overall, our test cases cover the majority of email delivery failure scenarios observed in the real world. Nevertheless, we acknowledge that our test cases are not exhaustive. Future work could leverage LLMs to systematically analyze relevant RFC documents and collect a broader range of real-world email delivery failures. This could enable the generation of more comprehensive test cases and provide a more complete understanding of email retry mechanisms.